

TERRAFORM TASK - 01

1. Install Terraform on your PC

Install Terraform on Your PC — Windows

1. Download Terraform

Open the official HashiCorp Terraform installation page:

[Terraform Installation](#)

2. Extract the ZIP file

After downloading, extract it.

You will get:

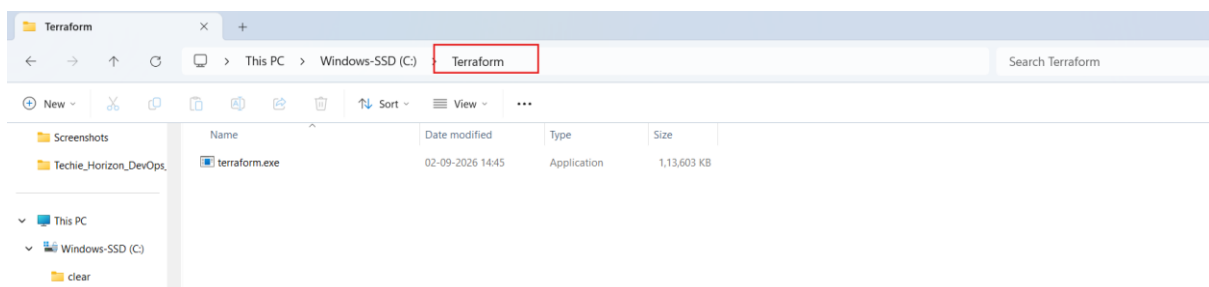
terraform.exe

Create a folder:

C:\Terraform

Move terraform.exe into:

C:\Terraform\terraform.exe



3. Add Terraform to PATH

1. Press **Windows + S**
2. Search for **Environment Variables**
3. Click **Edit the system environment variables**
4. Click **Environment Variables**
5. Under **System variables**, select **Path**

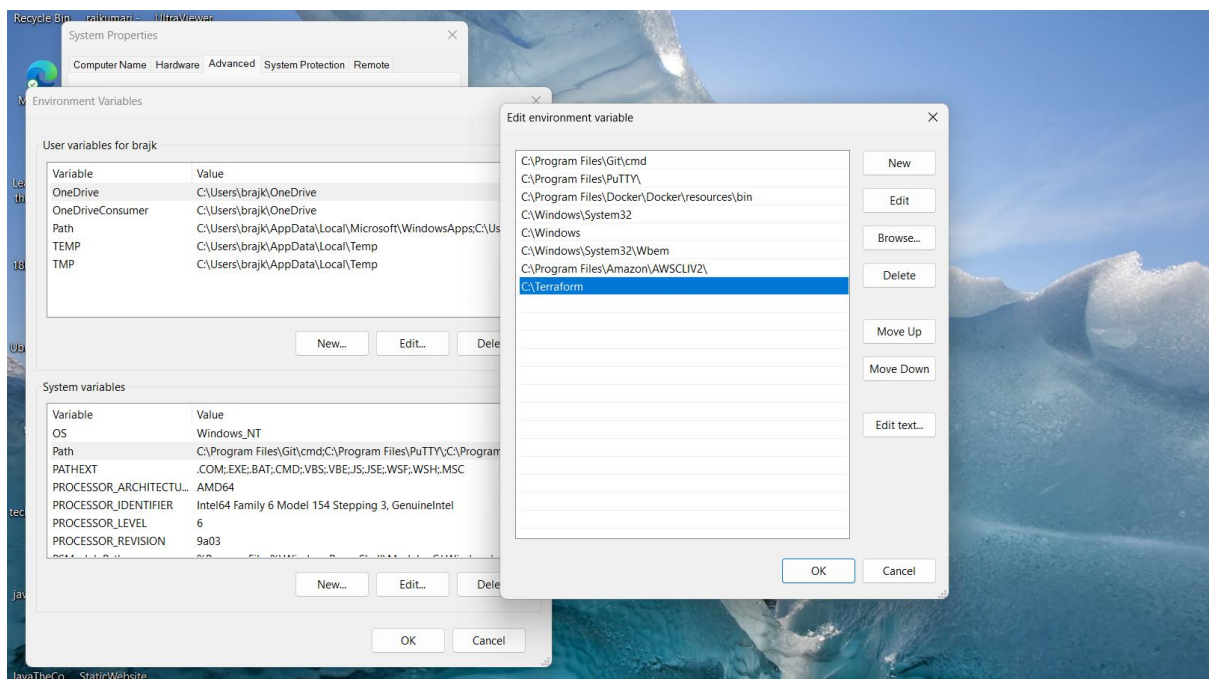
6. Click **Edit**

7. Click **New**

8. Add:

C:\Terraform

9. Click **OK** on all windows.



4. Verify Terraform installation

Open a new **Command Prompt** or PowerShell window or git bash

Run: terraform version and where terraform

```
MINGW64:/c/Terraform  
  
brajk@RajKumari MINGW64 /c/Terraform  
$ terraform version  
Terraform v1.16.0  
on windows_386  
  
brajk@RajKumari MINGW64 /c/Terraform  
$ where terraform  
C:\Terraform\terraform.exe  
  
brajk@RajKumari MINGW64 /c/Terraform  
$ |
```

2. Execute all the templates shown in video.

First Template is main.tf using local provider

The screenshot shows the Terraform Registry page for the 'local' provider. The page has a dark header with the Terraform logo and 'Registry' text. Below the header is a search bar. The main content area shows the provider details for 'local' (hashicorp/local), including its description 'Used to manage local resources, such as creating files', version '2.9.0 (latest)', and a 'Utility' tag. At the bottom, there are tabs for 'Overview' and 'Documentation', with 'Documentation' being the active tab and highlighted with a red box.

Terraform | Registry

Search providers, modules, run tasks, and policies...

Providers / hashicorp / local / v2.9.0

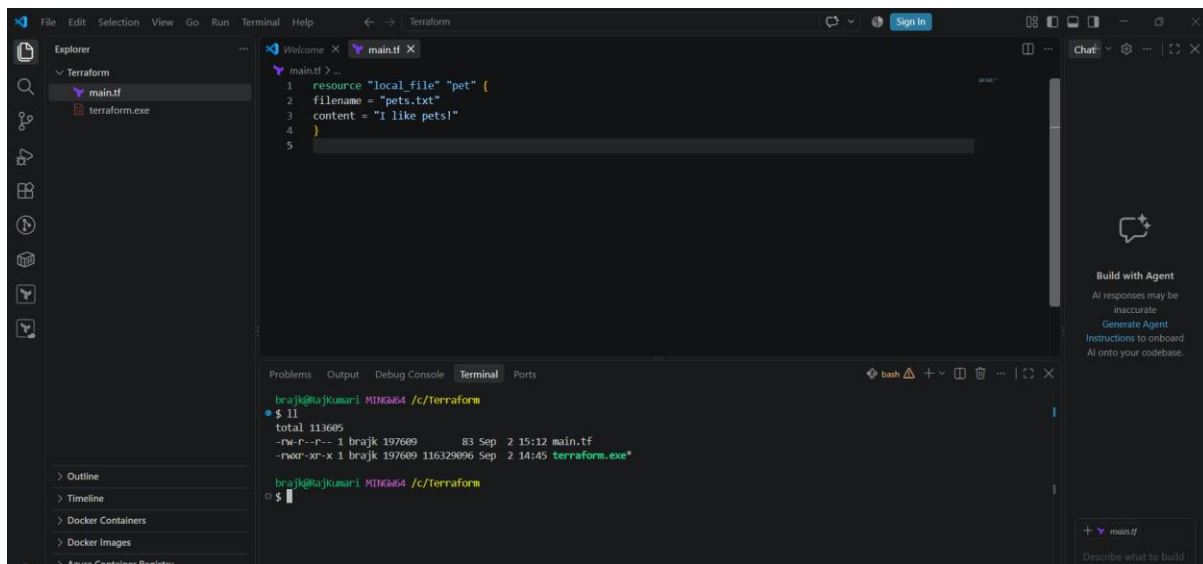
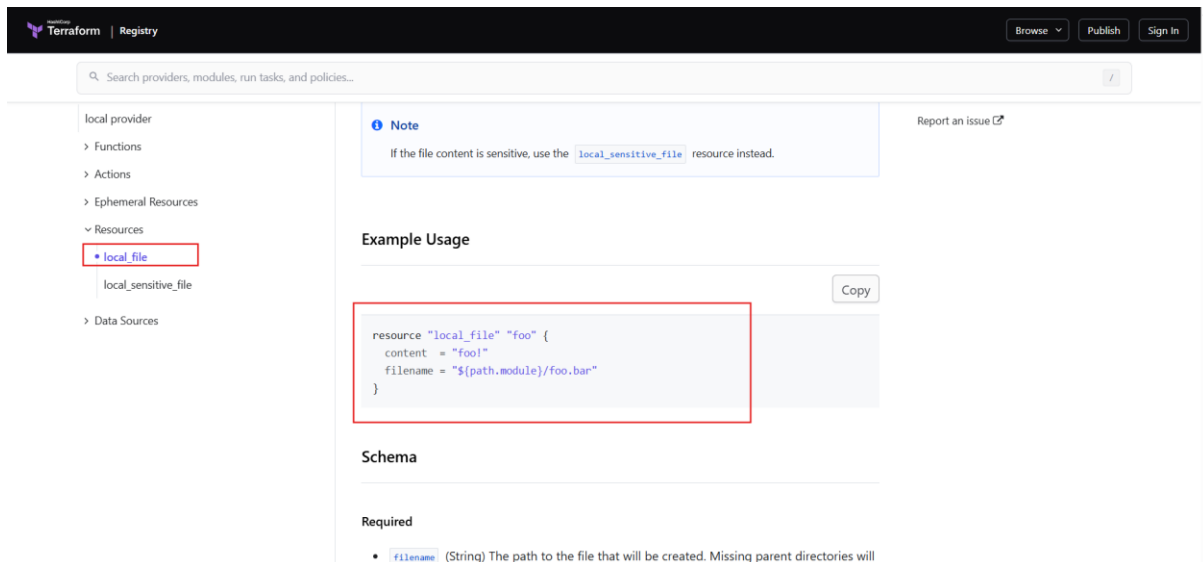
local Official Version 2.9.0 (latest) Vi

hashicorp/local
Used to manage local resources, such as creating files

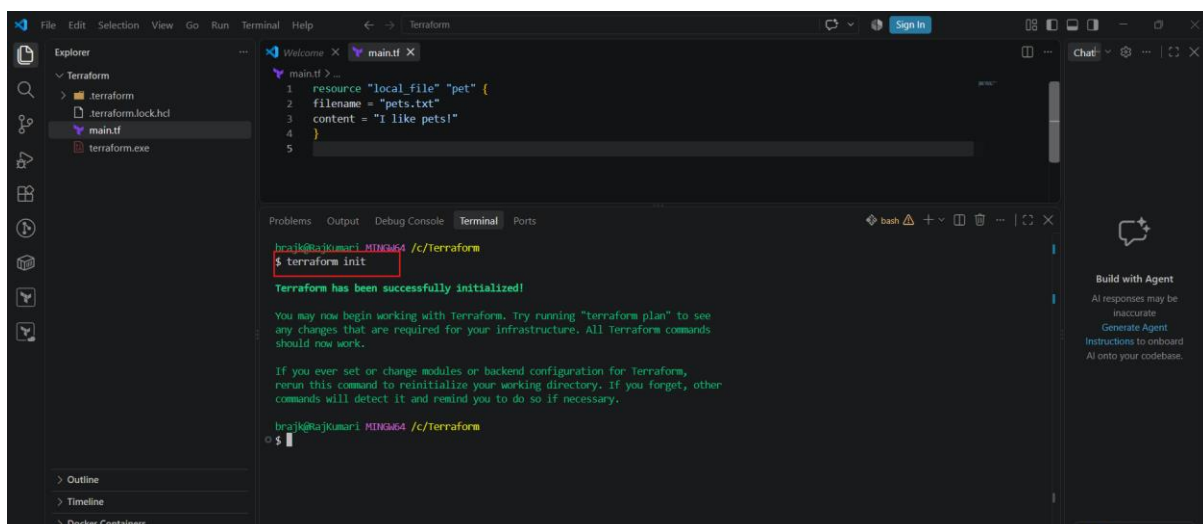
Downloads: 1.28 This week: 6.7M Versions: 27 Source code: hashicorp/local Published: May 13, 2026 Published by: HashiCorp

Utility

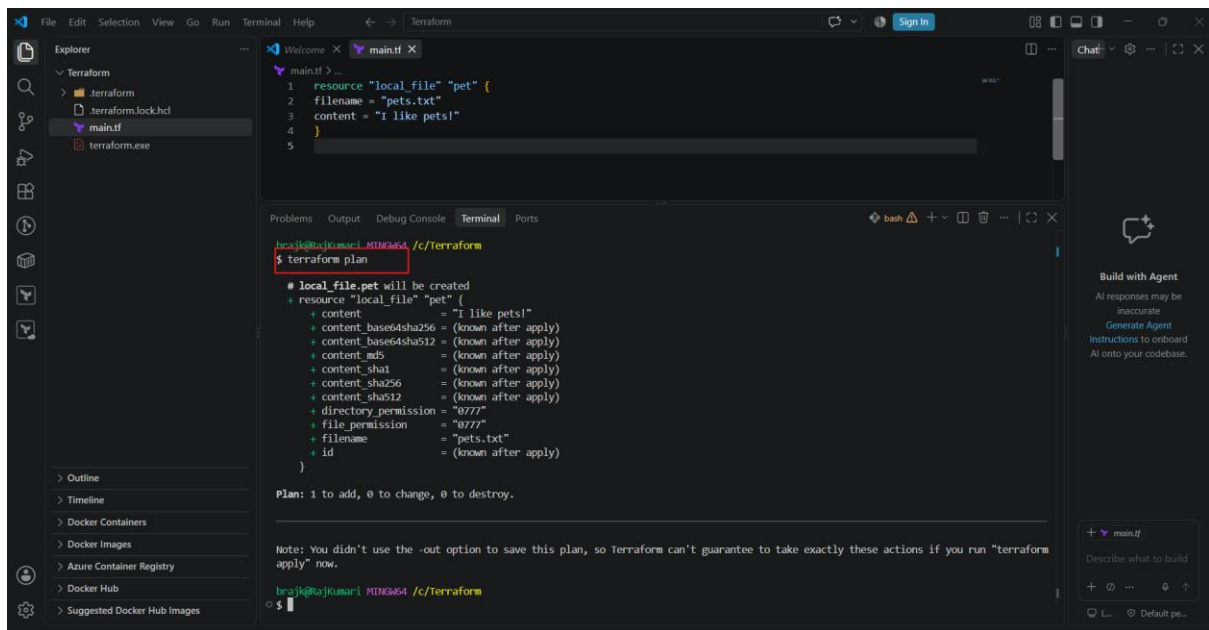
Overview Documentation



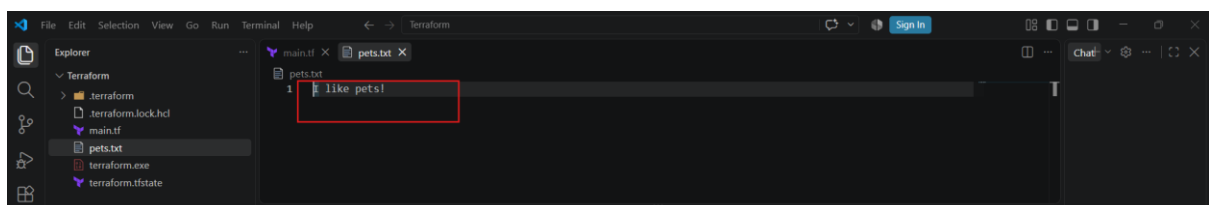
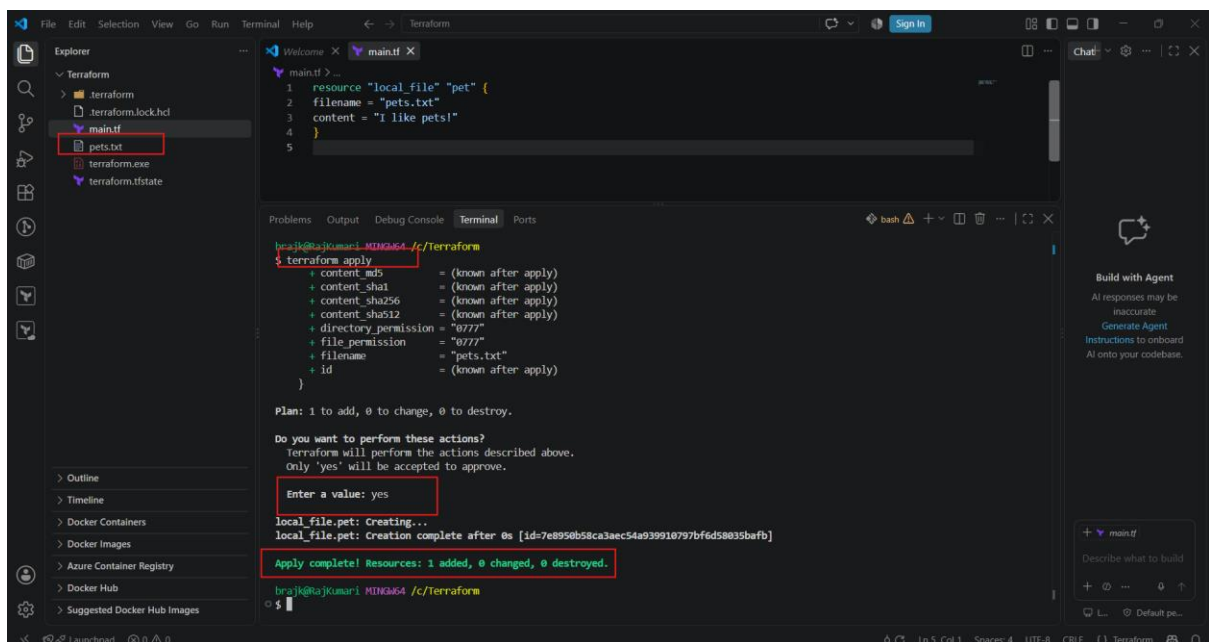
Command: \$ terraform init



Command: `$ terraform plan`



Command: \$ terraform apply



Modified main.tf file

```
1 resource "local_file" "pet" {
2   filename = "pets.txt"
3   content = "Pets are Faithful!!!!!!"
4 }
5
```

```
brajk@brajkumari-MINGW64 /c/Terraform
$ terraform plan
Terraform will perform the following actions:

# local_file.pet must be replaced
./ resource "local_file" "pet" {
  ~ content      = "I like pets!" -> "Pets are Faithful!!!!!!" # forces replacement
  ~ content_base64sha256 = "SnsitEbtHysurjZC/dm1N8kLZ11/8Kc fppWVZomG4-" -> (known after apply)
  ~ content_base64sha512 = "V4uUUDU7pZxG30buYjYhP4uK0G0cPU7fFPEKQ31YBCb+/FND111+tz73/fZSDFuMSEFGdI732xUAb018eijUQ==" -> (known after apply)
  ~ content_md5      = "c952d655d5b588b07004637b4640e34" -> (known after apply)
  ~ content_sha1      = "7e8950b58ca3aec54a939910797bf6d58035bafb" -> (known after apply)
  ~ content_sha256     = "4a7b2b446c7b4dcac988490bf0e620d341d8d65ffc3027e9a58598668986a" -> (known after apply)
  ~ content_sha512     = "578fae2140d44e96711b7d1bb29721cf6d47b41b470f53b16b3c4290df50a109bfbf1a0bf52f5ad67b27f7d948316e9b911f19d1fbd3c6e541a35f1e8a3b9" -> (known after apply)
  ~ id                = "7e8950b58ca3aec54a939910797bf6d58035bafb" -> (known after apply)
  # (3 unchanged attributes hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

brajk@brajkumari-MINGW64 /c/Terraform
```

```
brajk@brajkumari-MINGW64 /c/Terraform
$ terraform apply
~ content_sha1      = "7e8950b58ca3aec54a939910797bf6d58035bafb" -> (known after apply)
~ content_sha256     = "4a7b2b446c7b4dcac988490bf0e620d341d8d65ffc3027e9a58598668986a" -> (known after apply)
~ content_sha512     = "578fae2140d44e96711b7d1bb29721cf6d47b41b470f53b16b3c4290df50a109bfbf1a0bf52f5ad67b27f7d948316e9b911f19d1fbd3c6e541a35f1e8a3b9" -> (known after apply)
~ id                = "7e8950b58ca3aec54a939910797bf6d58035bafb" -> (known after apply)
# (3 unchanged attributes hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.pet: Destroying... [id=7e8950b58ca3aec54a939910797bf6d58035bafb]
local_file.pet: Destruction complete after 0s
local_file.pet: Creating...
local_file.pet: Creation complete after 0s [id=bf184ff4085ae9db87e59019d1cd2999e79ad43c]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

brajk@brajkumari-MINGW64 /c/Terraform
$
```

Command: \$ terraform destroy

The screenshot shows a VS Code editor with a Terraform configuration file named `main.tf` open. The file contains a `resource "local_file" "pet"` block with the following properties:

```
1 resource "local_file" "pet" {
2   filename = "pets.txt"
3   content = "Pets are Faithfulllllllll"
4 }
5
```

The terminal window shows the command `$ terraform destroy` being executed. The output displays the plan for destroying the resource `local_file.pet` and confirms its destruction.

```
braj@braj-kumar: ~/c/terraform
$ terraform destroy
- content_sha1      = "bf184ff4085ae9db87e59019d1cd2999e79ad43c" -> null
- content_sha256    = "a9de7d70e59f60d9274904918abb4d1e87f131b15ad7407501edadide836625" -> null
- content_sha512    = "dfec834475f7e5e821f0844a7b62f4b0dc922ac400d51aa561c00b0de51aa61333ac17592b170ec20156c3dfef7e8d923cc17ceca8233cc707bf34522e120022" -> null
- directory_permission = "0777" -> null
- file_permission    = "0777" -> null
- filename          = "pets.txt" -> null
- id                = "bf184ff4085ae9db87e59019d1cd2999e79ad43c" -> null

Plan: 0 to add, 0 to change, 1 to destroy.

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

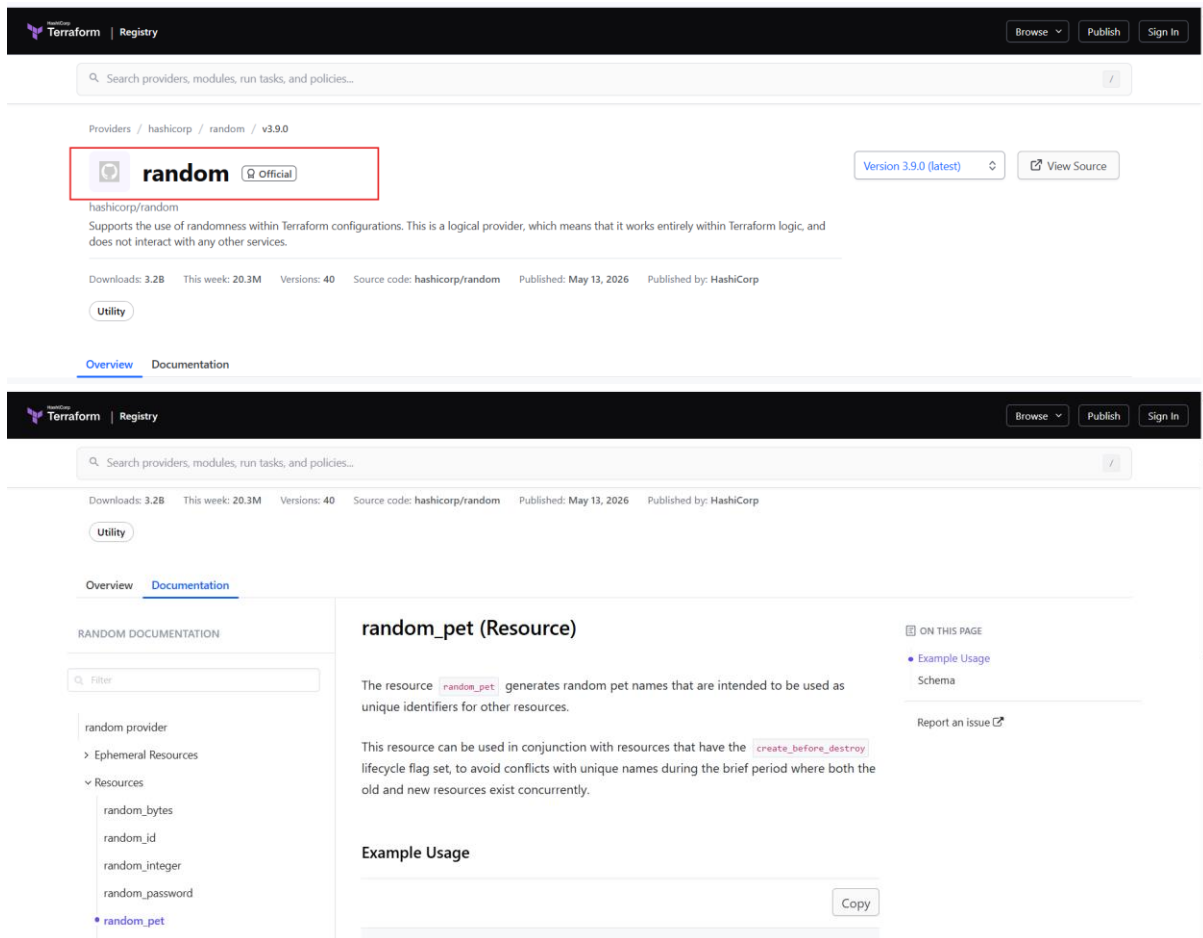
Enter a value: yes

local_file.pet: Destroying... [id=bf184ff4085ae9db87e59019d1cd2999e79ad43c]
local_file.pet: Destruction complete after 0s

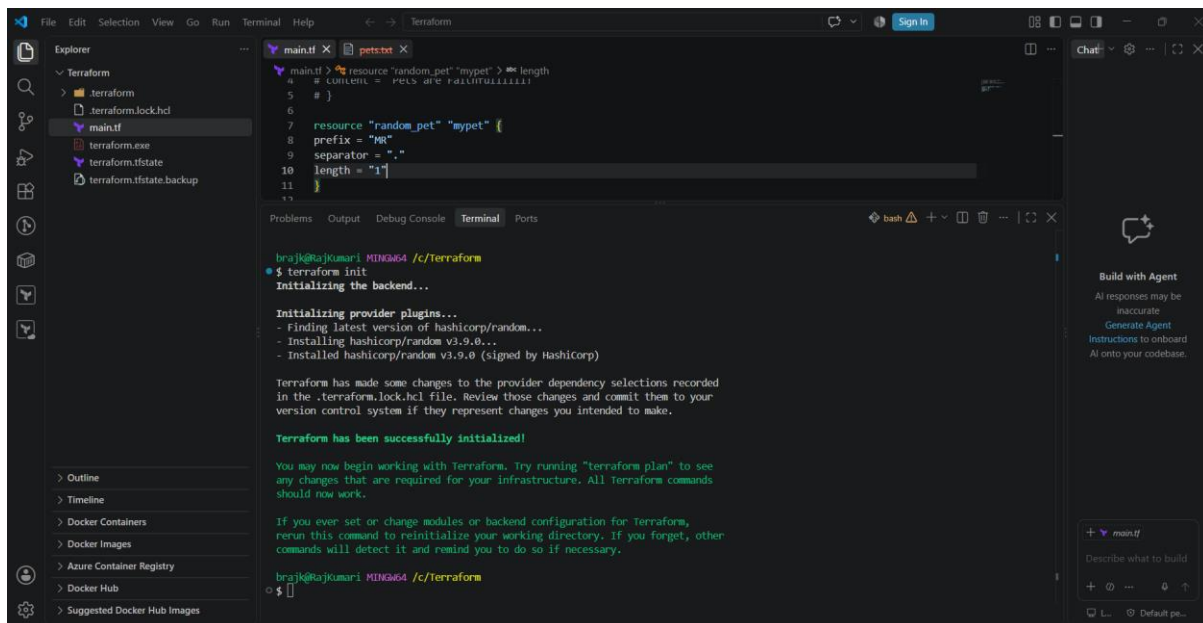
Destroy complete! Resources: 1 destroyed.

braj@braj-kumar: ~/c/terraform
$
```

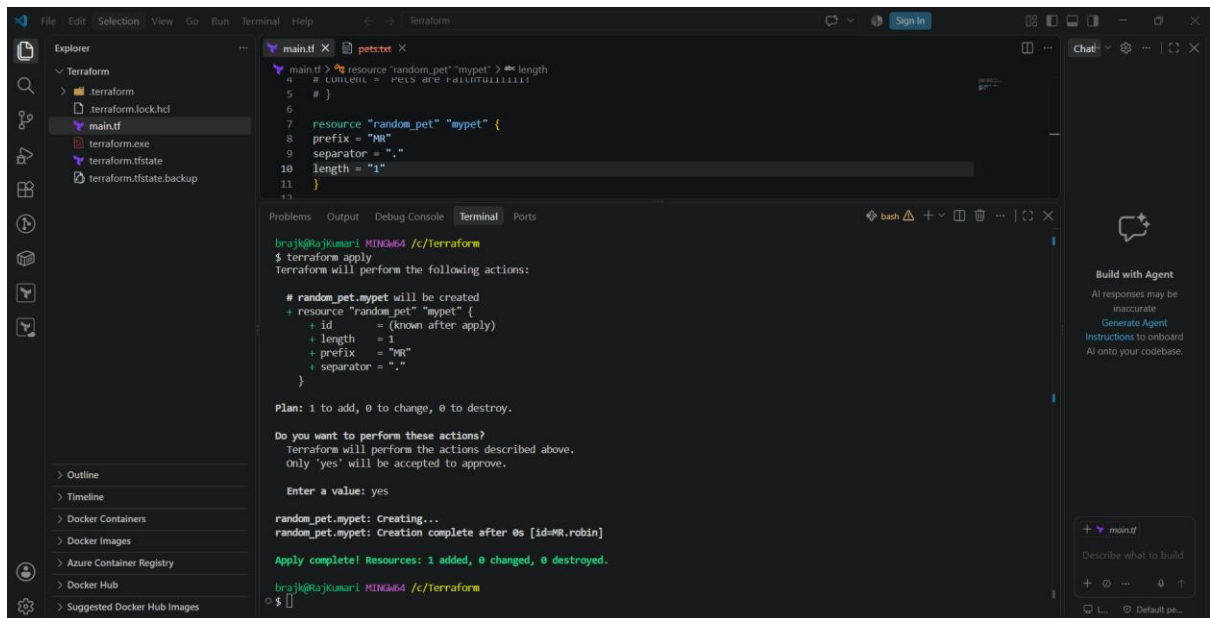
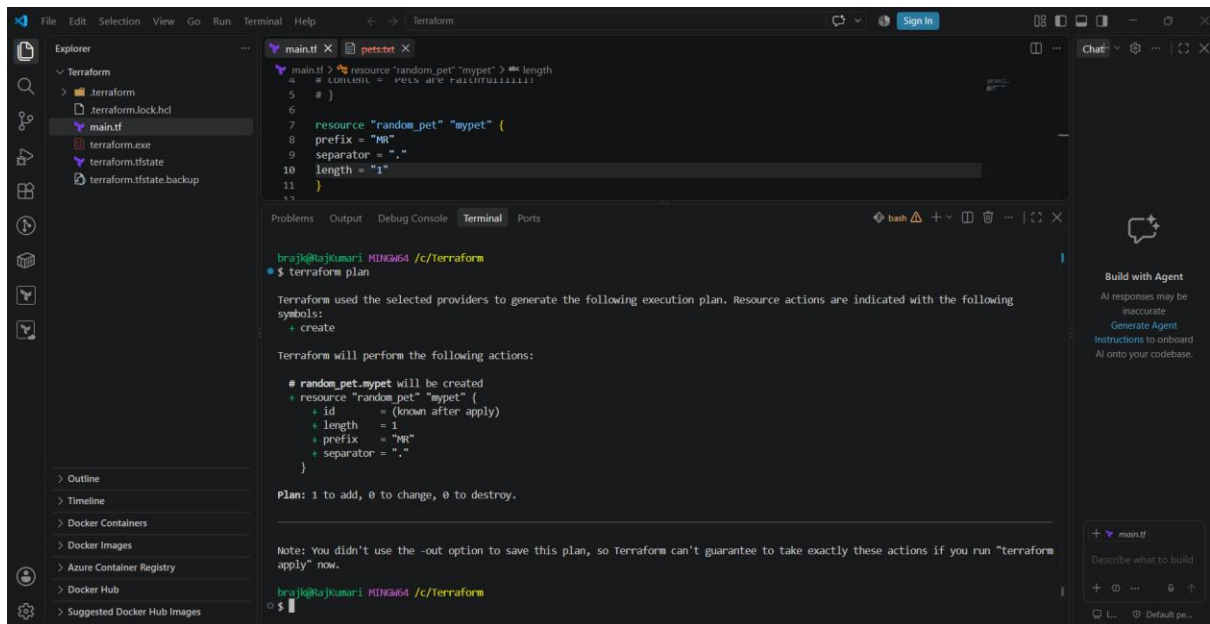
Second Template is main.tf using random provider

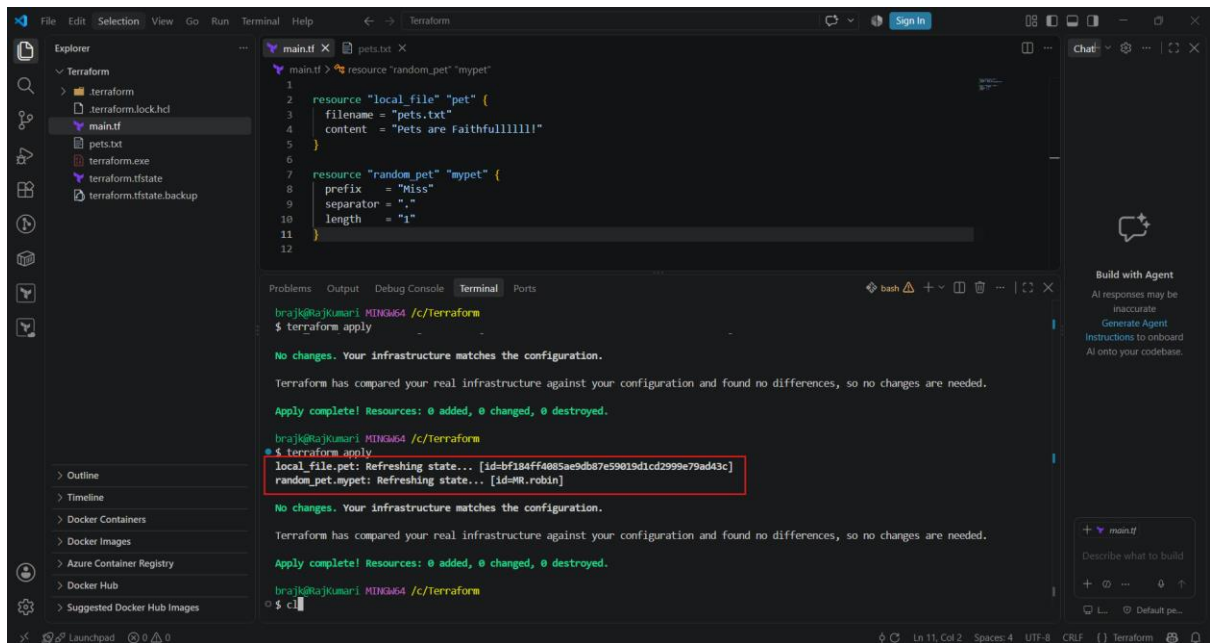
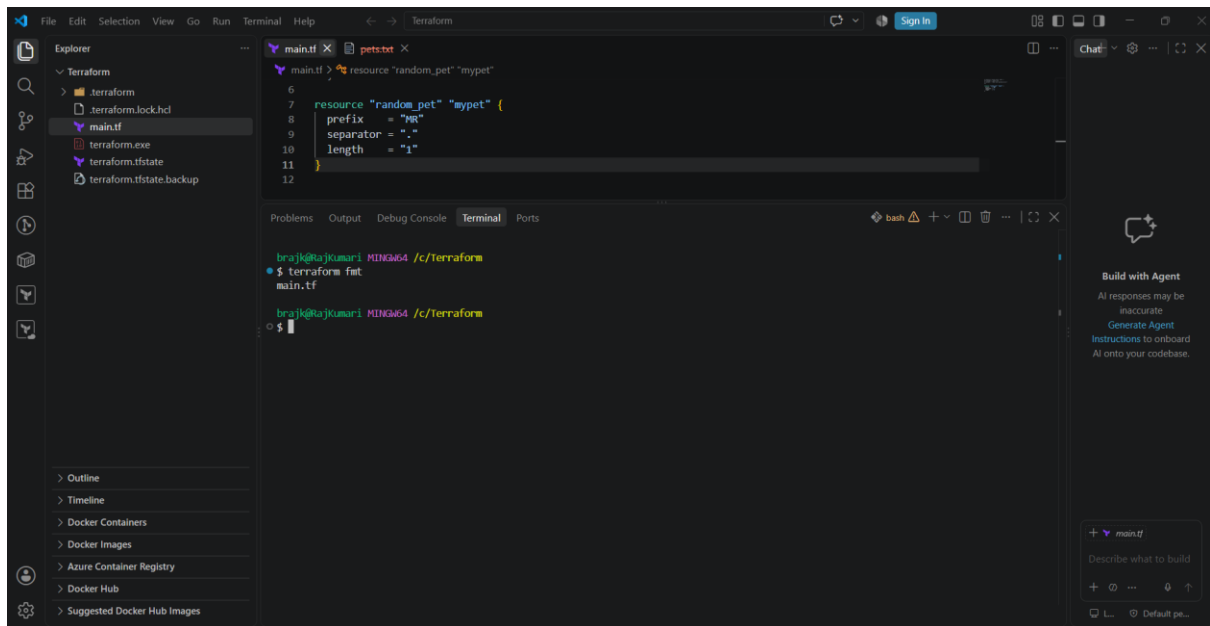


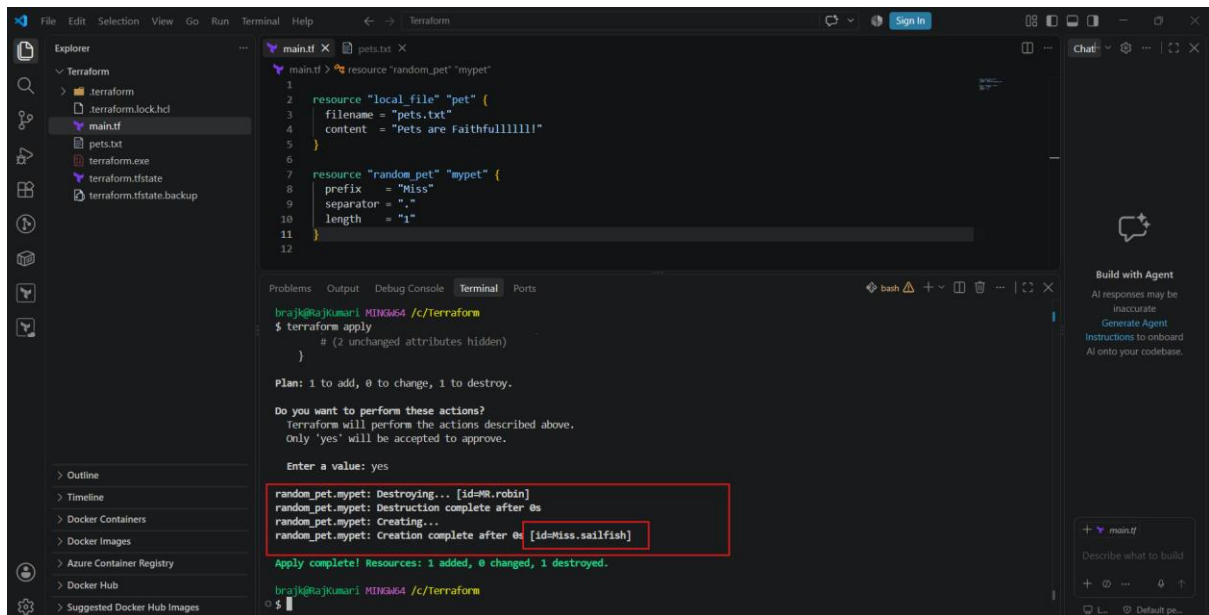
For Every provider, we need to initialise first. So, the command is “terraform init”



Command: \$ terraformfmt \$ terraform plan, \$ terraform apply, \$ terraform destroy







Terraform Command

Description

terraform init Initializes a Terraform working directory. It downloads the required **providers**, modules, and prepares the backend for Terraform state management.

terraform plan Creates an execution plan and shows what Terraform **will create, modify, or destroy** without actually making changes to the infrastructure.

terraform apply Executes the Terraform plan and **creates, updates, or deletes infrastructure** according to the configuration files.

terraform provider Used to manage and inspect Terraform **providers**. Providers allow Terraform to communicate with platforms such as AWS, Azure, GCP, Kubernetes, etc.

Example workflow

terraform init

terraform plan

terraform apply

Easy way to remember

init → **Prepare**

plan → **Preview**

apply → **Execute**

provider → **Connect to cloud/services**

For example, when using AWS:

```
provider "aws" {  
  
    region = "us-east-1"  
  
}
```

Here, the **AWS provider** allows Terraform to communicate with AWS and manage resources such as EC2, VPC, S3, and RDS.

Hands-on terraform Provider:

Step 1: Create a project folder

In VS Code:

File → **Open Folder** → **New Folder**

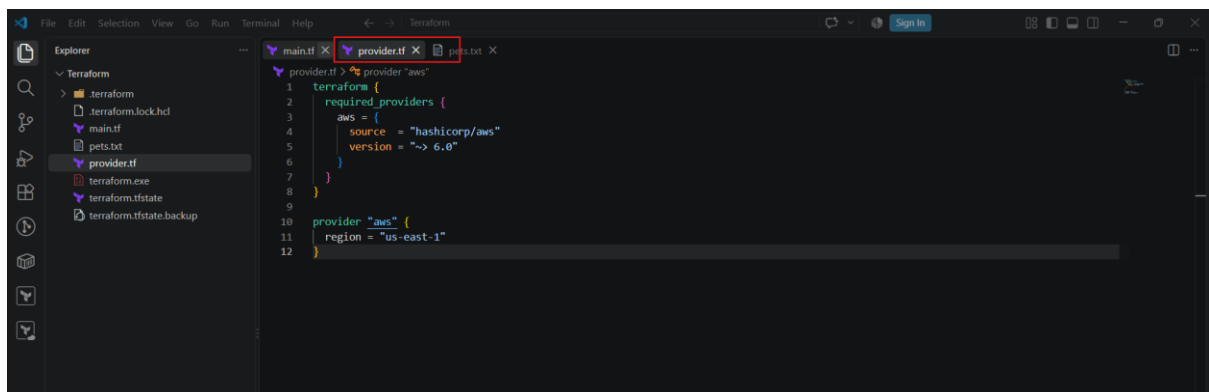
Create:

terraform-provider-demo

Step 2: Create provider.tf

In the VS Code Explorer, create:

provider.tf



Step 3: Create main.tf

Create another file:

main.tf

For a very simple provider test, we'll use the AWS caller identity data source:

```
data "aws_caller_identity" "current" {}
```

This **does not create an AWS resource**. It simply checks whether Terraform can communicate with AWS using the provider.

Step 3: Open VS Code Terminal

Click:

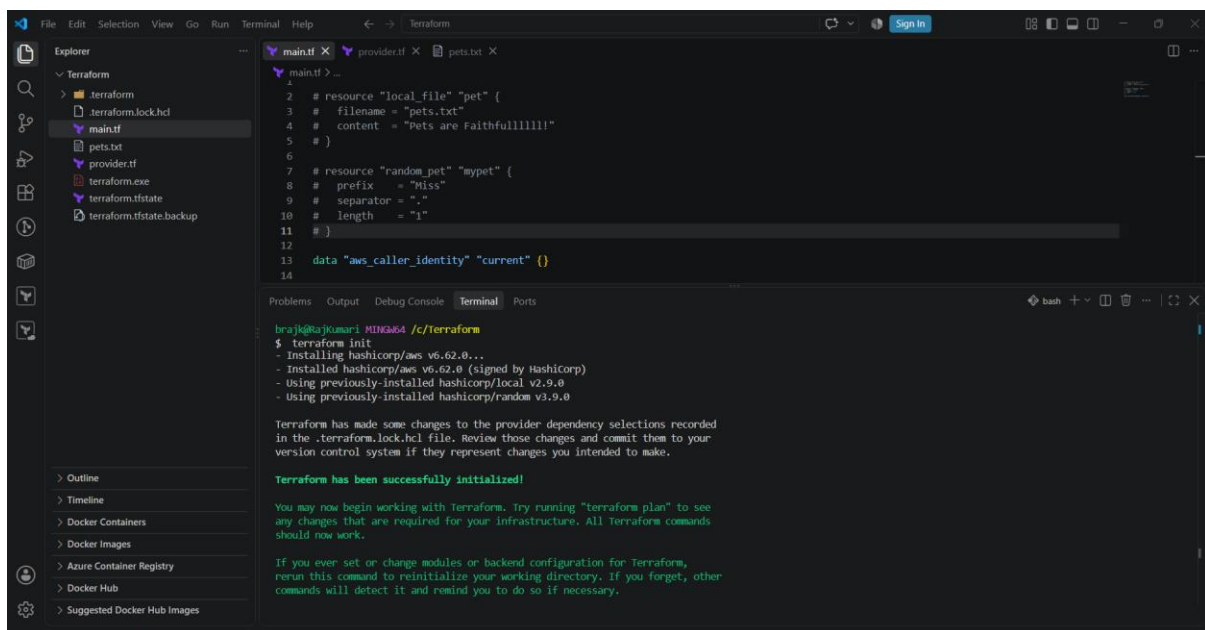
Terminal → **New Terminal**

You should see something like:

C:\Users\YourName\terraform

Step 4: Initialize Terraform

Run: `$ terraform init`



```
File Edit Selection View Go Run Terminal Help
main.tf provider.tf pets.txt
main.tf > ...
1 # resource "local_file" "pet" {
2 #   filename = "pets.txt"
3 #   content  = "Pets are Faithful!!!!!!!"
4 # }
5
6
7 # resource "random_pet" "mypet" {
8 #   prefix   = "Miss"
9 #   separator = "-"
10 #   length  = "1"
11 # }
12
13 data "aws_caller_identity" "current" {}
14

Problems Output Debug Console Terminal Ports
brajk@brajkumari MINGW64 /c/terraform
$ terraform init
- Installing hashicorp/aws v6.62.0...
- Installed hashicorp/aws v6.62.0 (signed by Hashicorp)
- using previously-installed hashicorp/local v2.9.0
- using previously-installed hashicorp/random v3.9.0

Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.

Terraform has been successfully initialized!

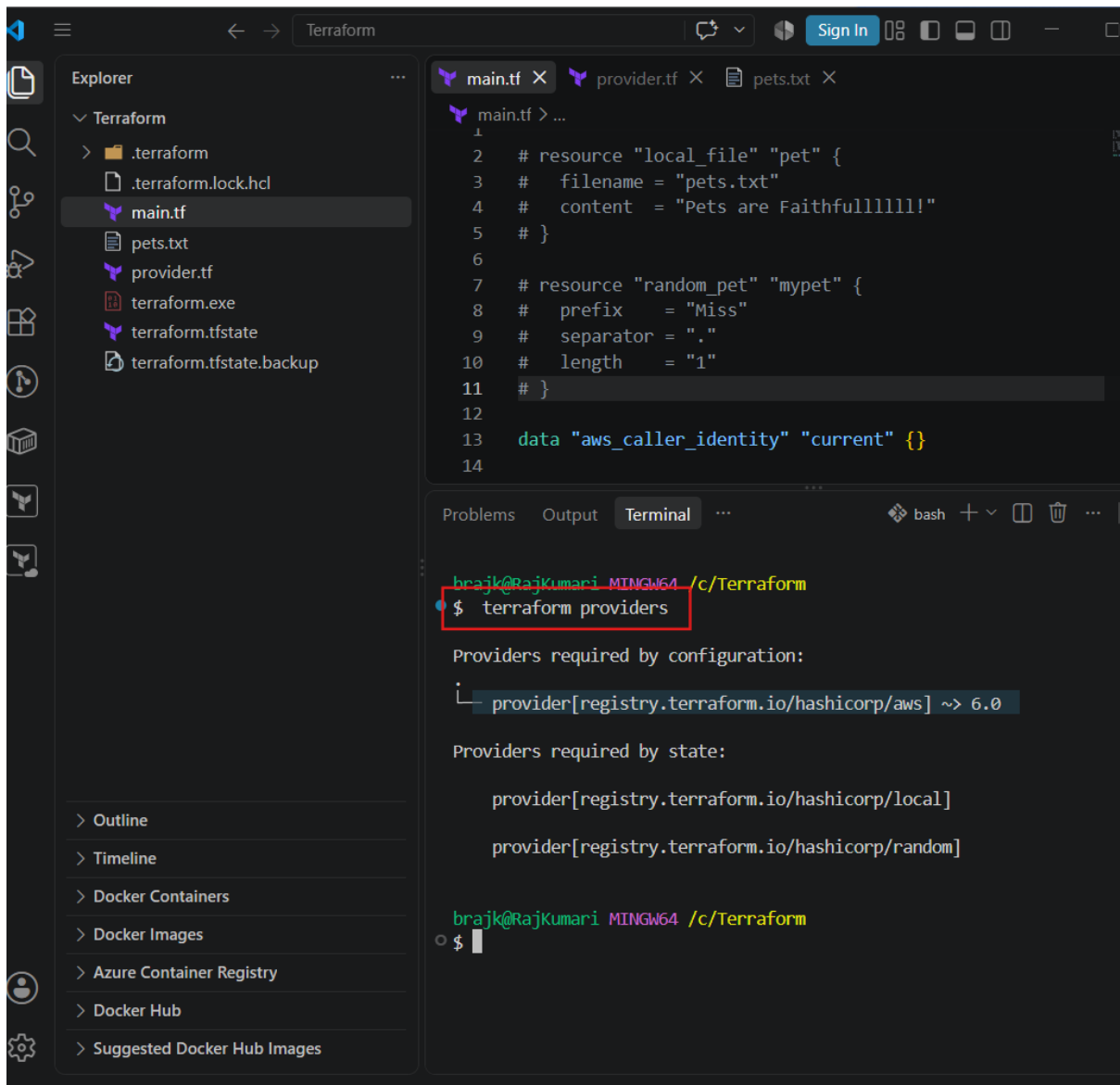
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Step 5: Check the provider

Run:

`terraform providers`



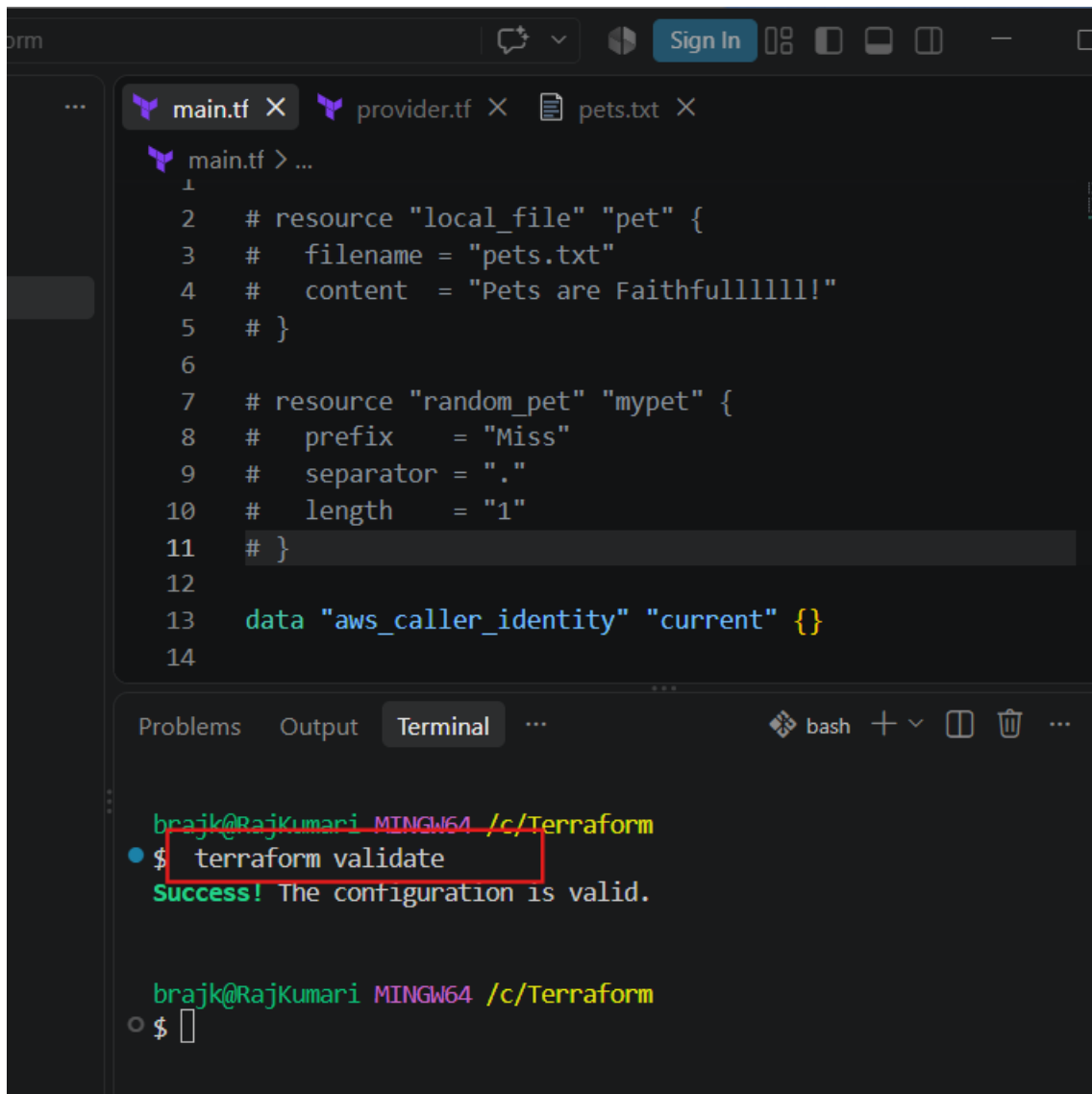
Step 7: Validate

Run:

terraform validate

Expected:

Success! The configuration is valid.



The screenshot shows a code editor with three tabs: `main.tf`, `provider.tf`, and `pets.txt`. The `main.tf` file contains the following Terraform configuration:

```
1
2 # resource "local_file" "pet" {
3 #   filename = "pets.txt"
4 #   content  = "Pets are Faithfulllllll!"
5 # }
6
7 # resource "random_pet" "mypet" {
8 #   prefix    = "Miss"
9 #   separator = "."
10 #   length   = "1"
11 # }
12
13 data "aws_caller_identity" "current" {}
14
```

Below the code editor is a terminal window with the following output:

```
brajk@RajKumari MINGW64 /c/Terraform
$ terraform validate
Success! The configuration is valid.

brajk@RajKumari MINGW64 /c/Terraform
$
```

Step 8: Run Plan

Run:

`terraform plan`

Because we're only checking AWS identity, Terraform should not plan to create an EC2, S3 bucket, etc.

Step 9: Apply

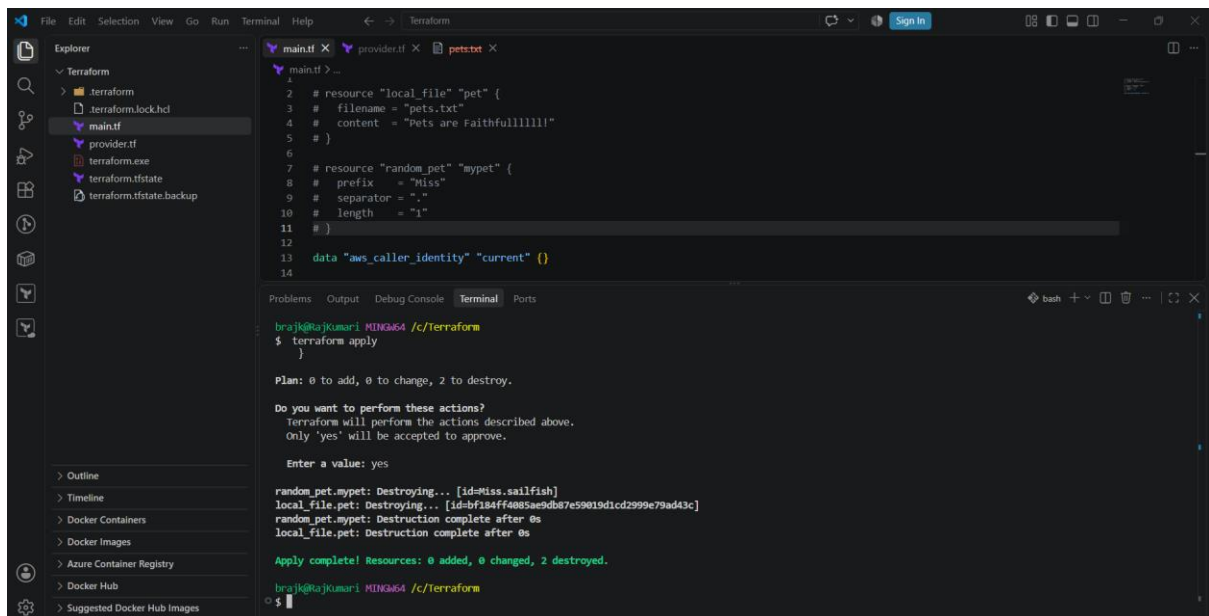
Run:

`terraform apply`

Type:

yes

Terraform will communicate with AWS using the AWS provider.



The screenshot shows a VS Code editor with a Terraform project. The Explorer panel on the left shows the file structure: `main.tf`, `provider.tf`, `terraform.exe`, `terraform.tfstate`, and `terraform.tfstate.backup`. The main editor displays the content of `main.tf`:

```
1 #
2 # resource "local_file" "pet" {
3 #   filename = "pets.txt"
4 #   content  = "Pets are Faithfullllllll!"
5 # }
6
7 # resource "random_pet" "mypet" {
8 #   prefix   = "Miss"
9 #   separator = "."
10 #   length  = "1"
11 # }
12
13 data "aws_caller_identity" "current" {}
14
```

The terminal window at the bottom shows the execution of `terraform apply`. It displays the plan (0 to add, 0 to change, 2 to destroy), a confirmation prompt, and the final output:

```
brajk@rajkumar: ~/c/terraform
$ terraform apply

Plan: 0 to add, 0 to change, 2 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

random_pet.mypet: Destroying... [id=Miss.sailfish]
local_file.pet: Destroying... [id=b7184ff4085a69db7e59019d1cd2999e79ad43c]
random_pet.mypet: Destruction complete after 0s
local_file.pet: Destruction complete after 0s

Apply complete! Resources: 0 added, 0 changed, 2 destroyed.

brajk@rajkumar: ~/c/terraform
$
```

4. Integrate a sample Terraform template in Jenkins.

The goal is:

GitHub



Jenkins



Terraform Init



Terraform Validate



Terraform Plan



Terraform Apply



AWS Resource

We'll use a simple **AWS S3 bucket** example.

Step 1: Create Terraform project

In your VS Code project, create:

terraform-jenkins-demo/

|

├── provider.tf

├── main.tf

└── Jenkinsfile

provider.tf

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 6.0"  
    }  
  }  
}
```

```
provider "aws" {  
  region = "us-east-1"  
}
```

main.tf

```
resource "aws_s3_bucket" "demo" {  
  bucket = "terraform-jenkins-demo-123456789"
```

```
}
```

⚠ S3 bucket names must be globally unique. Change the bucket name if necessary.

Step 2: Test Terraform locally

From the VS Code terminal:

```
terraform init
```

Then:

```
terraform validate
```

Then:

```
terraform plan
```

If everything is successful, you can move to Jenkins.

Step 3: Create a GitHub repository

Create a repository such as:

```
terraform-jenkins-demo
```

Push your files:

```
provider.tf
```

```
main.tf
```

```
Jenkinsfile
```

Your GitHub repository should look like:

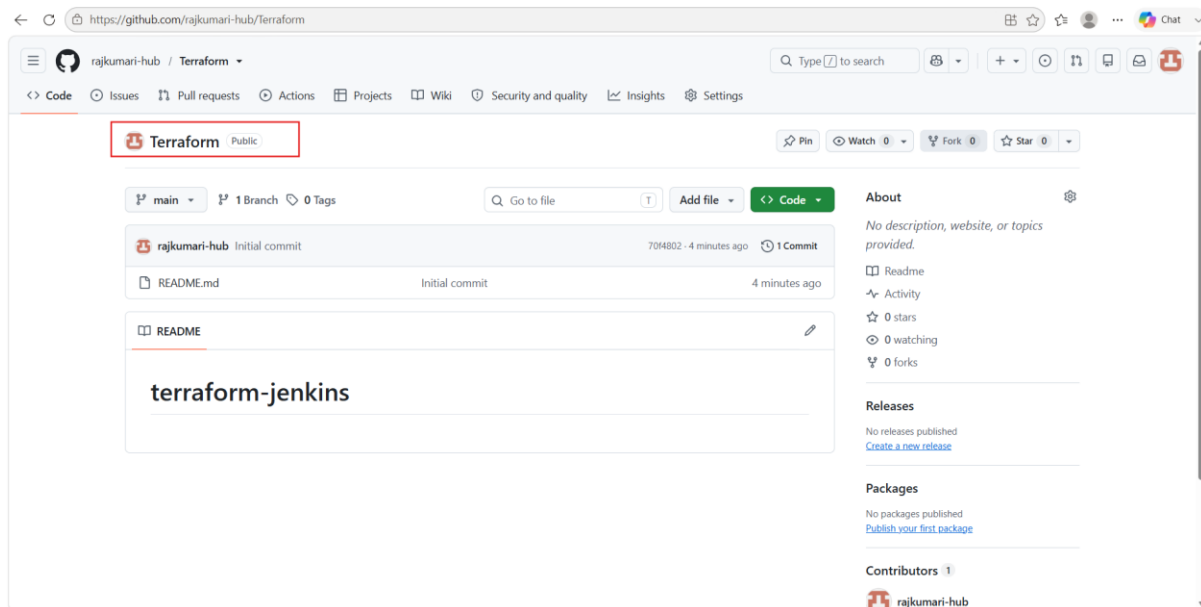
```
terraform-jenkins-demo
```

```
|
```

```
├── provider.tf
```

```
├── main.tf
```

```
└── Jenkinsfile
```



```
MINGW64:/c/Terraform

brajk@RajKumari MINGW64 /c/Terraform
$ git clone git@github.com:rajkumari-hub/Terraform.git
Cloning into 'Terraform'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

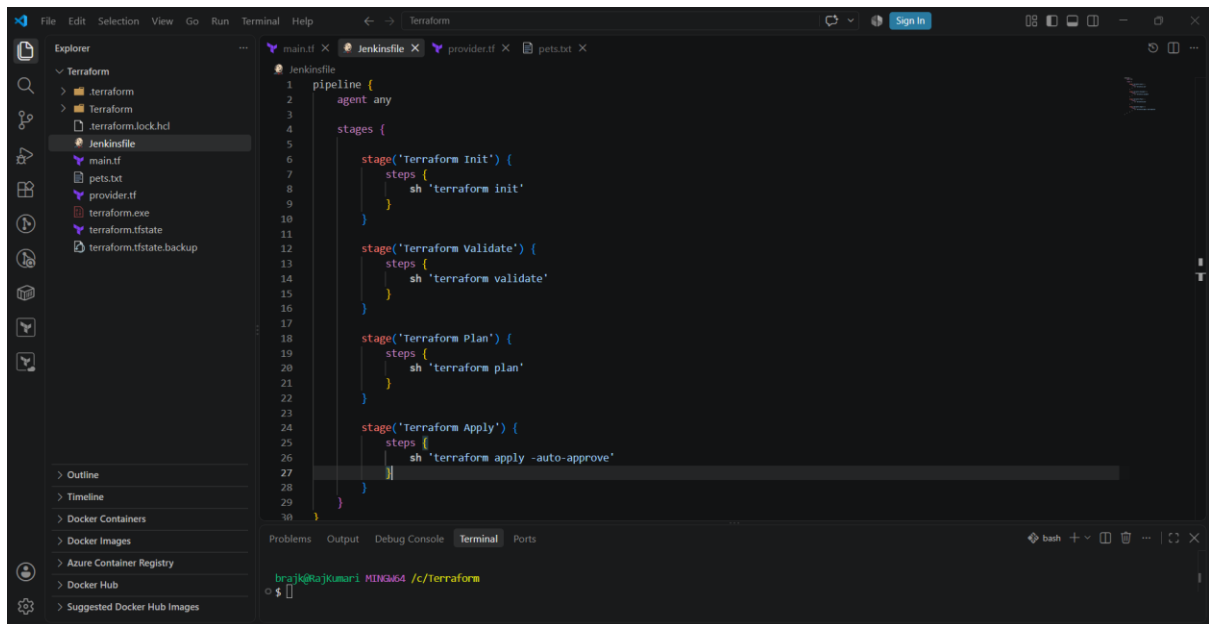
brajk@RajKumari MINGW64 /c/Terraform
$ pwd
/c/Terraform

brajk@RajKumari MINGW64 /c/Terraform
$ .....
```

Step 4: Create Jenkinsfile

Create a file named:

Jenkinsfile



What each stage does

Stage	Purpose
Terraform Init	Downloads providers and initializes Terraform
Terraform Validate	Checks Terraform syntax/configuration
Terraform Plan	Shows what Terraform will create/change
Terraform Apply	Creates/updates the AWS infrastructure

Step 5: Install Terraform on Jenkins

Your Jenkins server must have Terraform installed.

On your Jenkins EC2/Linux server, check:

terraform version

If you get:

terraform: command not found

Terraform isn't installed.

After installation, verify:

terraform version

For this example, the role needs appropriate S3 permissions.

Then test from the Jenkins server:

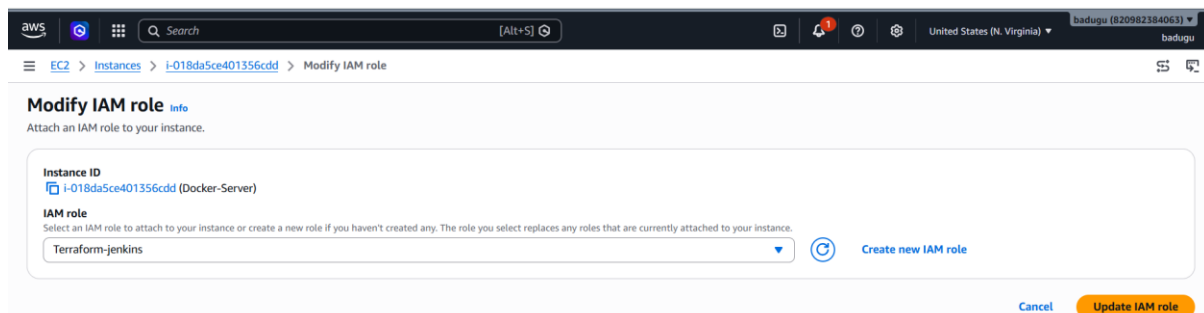
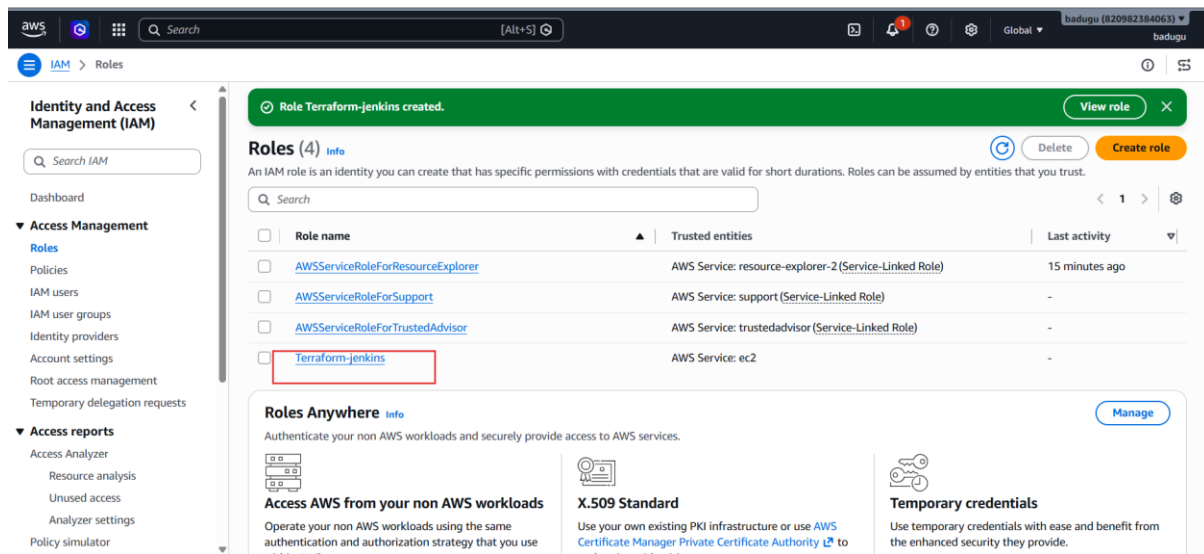
```
aws sts get-caller-identity
```

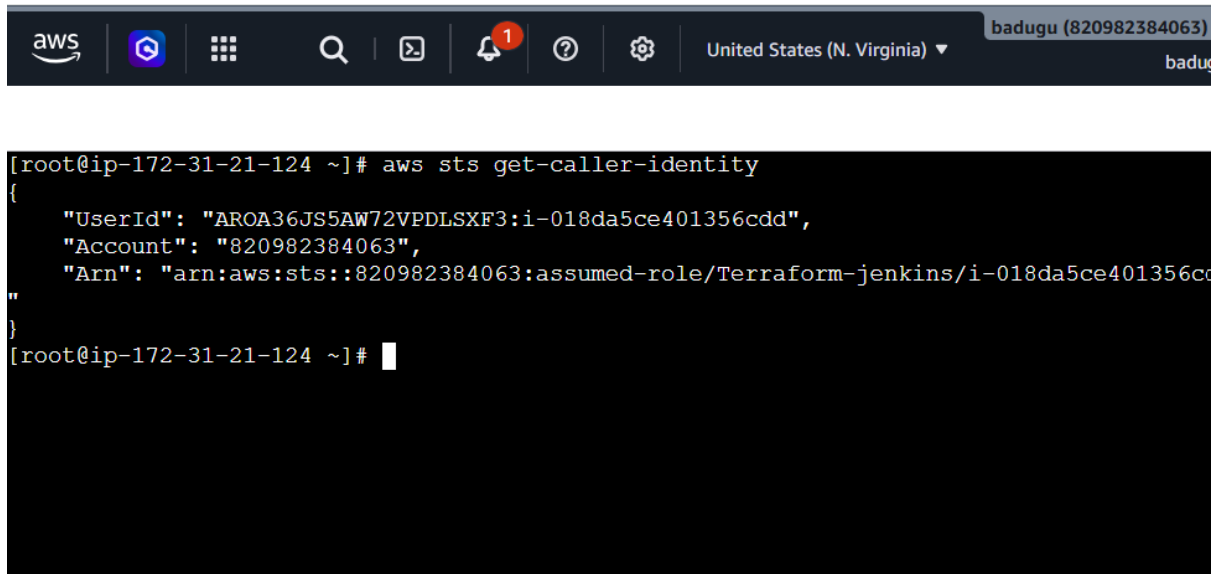
You should get something like:

Account: 123456789012

Arn: arn:aws:iam::123456789012:role/JenkinsRole

This confirms Jenkins can authenticate with AWS.





```
[root@ip-172-31-21-124 ~]# aws sts get-caller-identity
{
  "UserId": "ARO36JS5AW72VPDLSXF3:i-018da5ce401356cdd",
  "Account": "820982384063",
  "Arn": "arn:aws:sts::820982384063:assumed-role/Terraform-jenkins/i-018da5ce401356cdd"
}
```

Step 7: Create Jenkins Pipeline

Open Jenkins:

Jenkins Dashboard



New Item



terraform-jenkins-demo

Choose:

Pipeline

Click **OK**.

Under **Pipeline**, choose:

Definition:

Pipeline script from SCM

Select:

SCM: Git

Enter your GitHub repository URL.

For example:

https://github.com/YOUR_USERNAME/terraform-jenkins-demo.git

Set:

Branch:

*/main

And:

Script Path:

Jenkinsfile

Click:

Save

← → ↻ ⚠ Not secure 54.198.178.66:8080/view/all/newJob ☆ b ⋮

Jenkins / All ▾ / New Item 🔍 ⚙️ 👤

New Item

Enter an item name

terraform-jenkins

Select an item type

- Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
- Multibranch Pipeline**
Creates a set of Pipeline projects according to detected branches in one SCM repository.

OK

← → ↻ ⚠ Not secure 54.198.178.66:8080/job/terraform-jenkins/configure ☆ b ⋮

 **Jenkins**

terraform-jenkins ▾ / Configure

🔍

⚙️

👤

Git ▾

Repositories ?

Repository URL ?

https://github.com/rajkumari-hub/Terraform.git

Credentials ?

- none - ▾ + Add

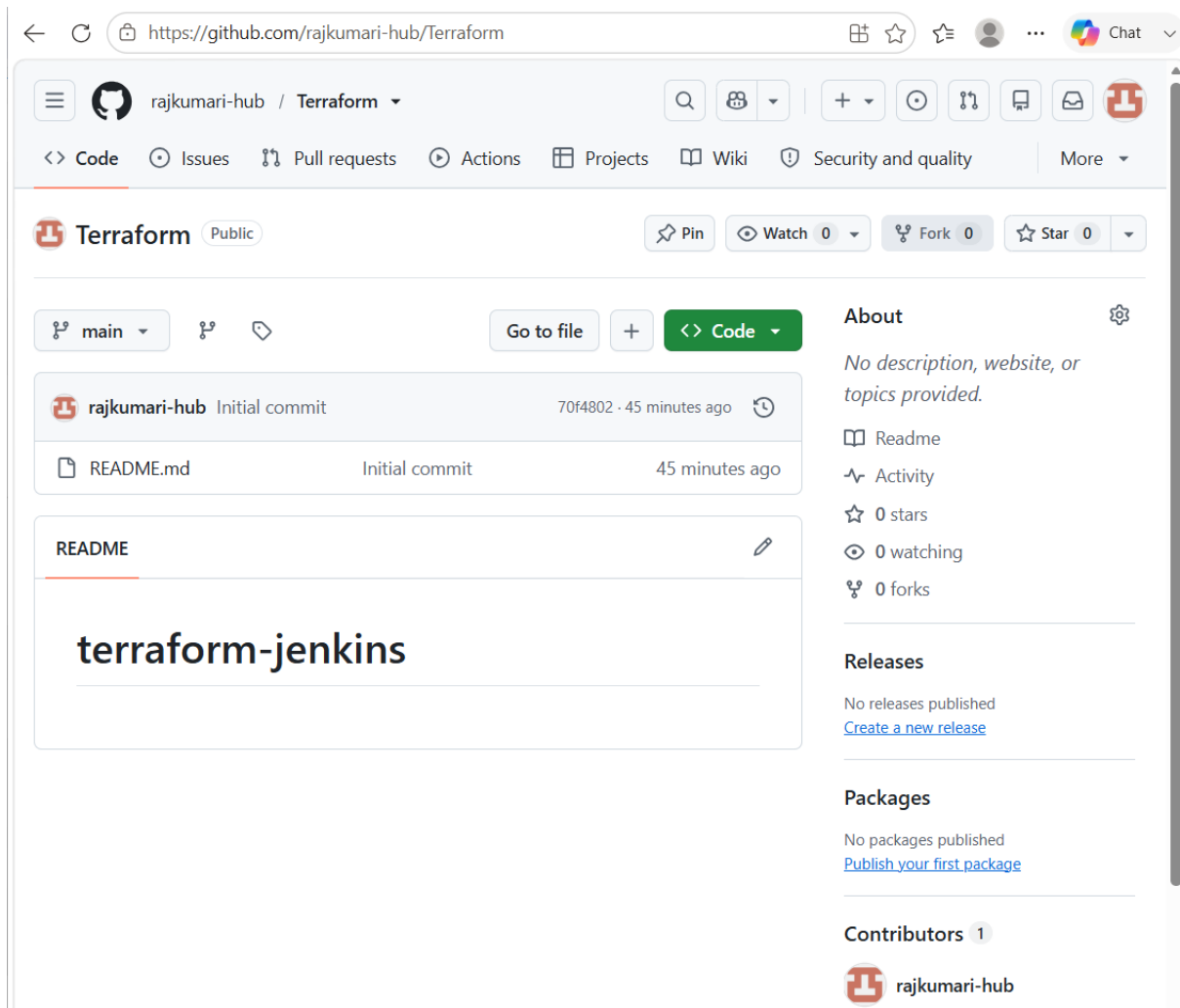
Advanced ▾

+ Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/main



Step 8: Run Jenkins Pipeline

Click:

Build Now

Jenkins will execute:

Checkout



Terraform Init



Terraform Validate



Terraform Plan



Terraform Apply

You can click:

Build #1



Console Output

A screenshot of the Visual Studio Code editor interface. The Explorer sidebar on the left shows a project structure with folders 'Terraform' and 'Jenkinsfile', and files 'main.tf', 'provider.tf', 'README.md', '.terraform.lock.hcl', 'pets.txt', 'terraform.exe', 'terraform.tfstate', and 'terraform.tfstate.backup'. The main editor area displays the 'Jenkinsfile' with a pipeline configuration. The pipeline has two stages: 'Terraform Plan' and 'Terraform Apply'. The 'Terraform Apply' stage has a step that runs 'sh 'terraform apply -auto-approve''. The bottom panel shows the 'Terminal' tab with a bash shell. The terminal output shows the user committing files to a git repository and pushing them to the origin main branch. The commit message is 'Addedd terraform files'. The push output shows the objects being enumerated, counted, compressed, and written to the repository.

```
1 pipeline {
4   stages {
17
18     stage('Terraform Plan') {
19       steps {
20         sh 'terraform plan'
21       }
22     }
23
24     stage('Terraform Apply') {
25       steps {
26         sh 'terraform apply -auto-approve'
27       }
28     }
29   }
30 }
```

```
brajk@rajkumari MINGW64 /c/Terraform/Terraform (main)
$ git commit -m "Addedd terraform files"
create mode 100644 Jenkinsfile
create mode 100644 main.tf
create mode 100644 provider.tf

brajk@rajkumari MINGW64 /c/Terraform/Terraform (main)
$ git push origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 824 bytes | 824.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:rajkumari-hub/terraform.git
70f4802..a10a925  main -> main

brajk@rajkumari MINGW64 /c/Terraform/Terraform (main)
```

← ↻ https://github.com/rajkumari-hub/Terraform

rajkumari-hub / Terraform

<> Code Issues Pull requests Actions Projects Wiki Security and quality More

Terraform Public

Pin Watch 0 Fork 0 Star 0

main

Go to file + <> Code

rajkumari-hub Added terraform files a10a925 · 1 minute ago

Jenkinsfile	Added terraform files	1 minute ago
README.md	Initial commit	52 minutes ago
main.tf	Added terraform files	1 minute ago
provider.tf	Added terraform files	1 minute ago

README

terraform-jenkins

About

No description, website, or topics provided.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Contributors 1

rajkumari-hub

← → ↻ Not secure 54.224.125.227:8080/job/terraform-jenkins/

Jenkins / terraform-jenkins

Status

</> Changes

▶ Build Now

🗑 Delete Pipeline

⚙ Configure

✎ Rename

📊 Stages

🔗 Pipeline Syntax

🟢 terraform-jenkins

Permalinks

- Last build (#2), 2 min 20 sec ago
- Last stable build (#2), 2 min 20 sec ago
- Last successful build (#2), 2 min 20 sec ago
- Last failed build (#1), 10 min ago
- Last unsuccessful build (#1), 10 min ago
- Last completed build (#2), 2 min 20 sec ago

Builds

Filter

Today

- 🟢 #3 2:03 PM
- 🟢 #2 2:01 PM

```

Jenkins / terraform-jenkins / #2

[32m+[0m logging (known after apply)

[32m+[0m object_lock_configuration (known after apply)

[32m+[0m replication_configuration (known after apply)

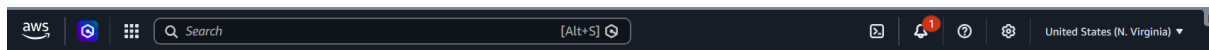
[32m+[0m server_side_encryption_configuration (known after apply)

[32m+[0m versioning (known after apply)

[32m+[0m website (known after apply)
}

[1mPlan:[0m [0m to add, 0 to change, 0 to destroy.
[0m[1maws_s3_bucket.demo: Creating...[0m[0m
[0m[1maws_s3_bucket.demo: Creation complete after 1s [id=terraform-jenkins-demo-123456789][0m
[0m[1m[32m
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.[0m
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```



```

[root@ip-172-31-29-243 ~]# aws s3 ls
2026-09-02 14:03:36 terraform-jenkins-demo-123456789
[root@ip-172-31-29-243 ~]#

```

us-east-1.console.aws.amazon.com/s3/home?region=us-east-1#

Amazon S3

Buckets

General purpose buckets | All AWS Regions | Directory buckets

General purpose buckets (1) Info

Copy ARN | Empty | Delete | Create bucket

Buckets are containers for data stored in S3.

Find buckets by name

Name	AWS Region	Creation date
terraform-jenkins-demo-123456789	US East (N. Virginia) us-east-1	September 2, 2026, 19:33:36 (UTC+05:30)

Account snapshot Info

Updated daily

[View dashboard](#)

Storage Lens provides visibility into storage usage and activity trends.

External access summary Info

Updated daily

External access findings help you identify bucket permissions that allow public access or access from other AWS accounts.

